Analyzing the Effect of Network Width on Graph Transformers

Progress review 1 (Явц 1) — weeks 1–5

Barsbold Bayarerdene

October 2026

The question

How does a graph transformer's width — its embedding dimension m — decide what it can learn, and how must width grow with graph size n?

- **Theory** gives width bounds for *representing* graph algorithms: one attention layer needs width ≈ (what each token must gather) — q-sparse averaging, Sanford et al. 2023; connectivity at depth $\log N$ needs width only just above $\sqrt{N}$, with no extra tokens (N = vertices + edges; Sanford et al. 2024).
- **Nobody has measured** the width a transformer needs to *learn* a graph task, how it scales with n, or how it trades against depth.

This thesis measures the **critical width** m^* — the smallest width that learns the task — and how it moves with n, depth, data and training.

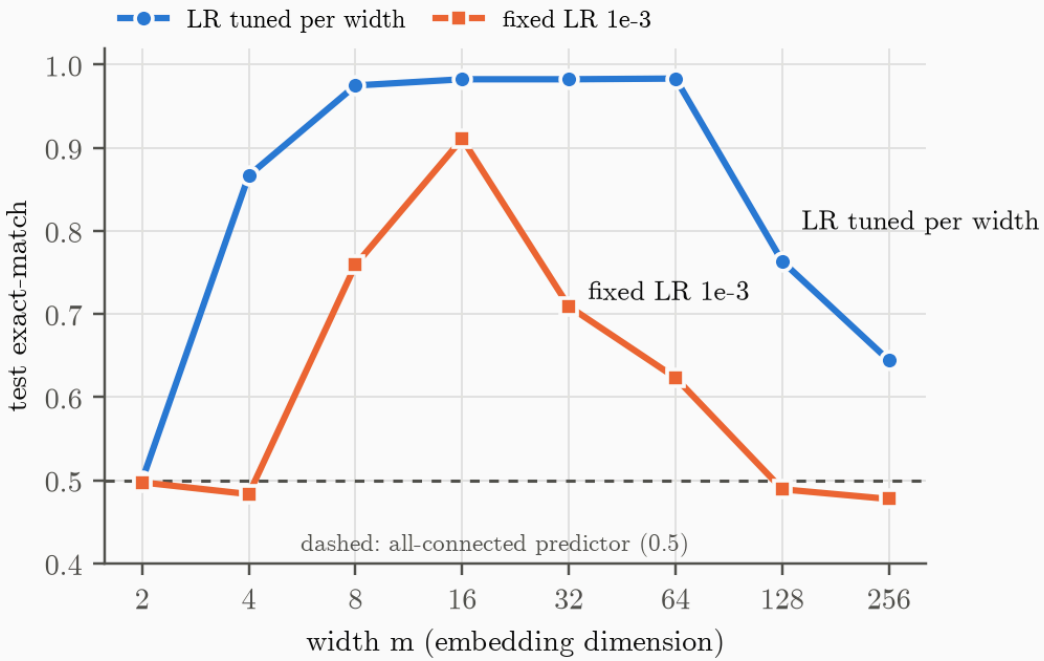
Set-up

- **Task:** connectivity matrix — for every pair of nodes, are they in the same component? (dense supervision; a single yes/no per graph does not train)
- **Model:** transformer encoder, one token per node (its adjacency row), pairwise read-out — task, tokens and read-out from Ye et al. (2026)
- **Width** m = embedding dimension; the whole model scales with it: $m/8$ attention heads of size 8, feed-forward layer $4m$
- **Metric:** pair accuracy (fraction of pairs right); trivial "all connected" ≈ 0.75

- **Critical width** m^***:** where accuracy crosses 0.95, per seed
- **Protocol:** choices on validation, results on test; 2–3 seeds
- **Compute:** sharded sweeps on Kaggle GPUs; $\approx$ 770 training runs so far

Part 1 — Measuring width honestly

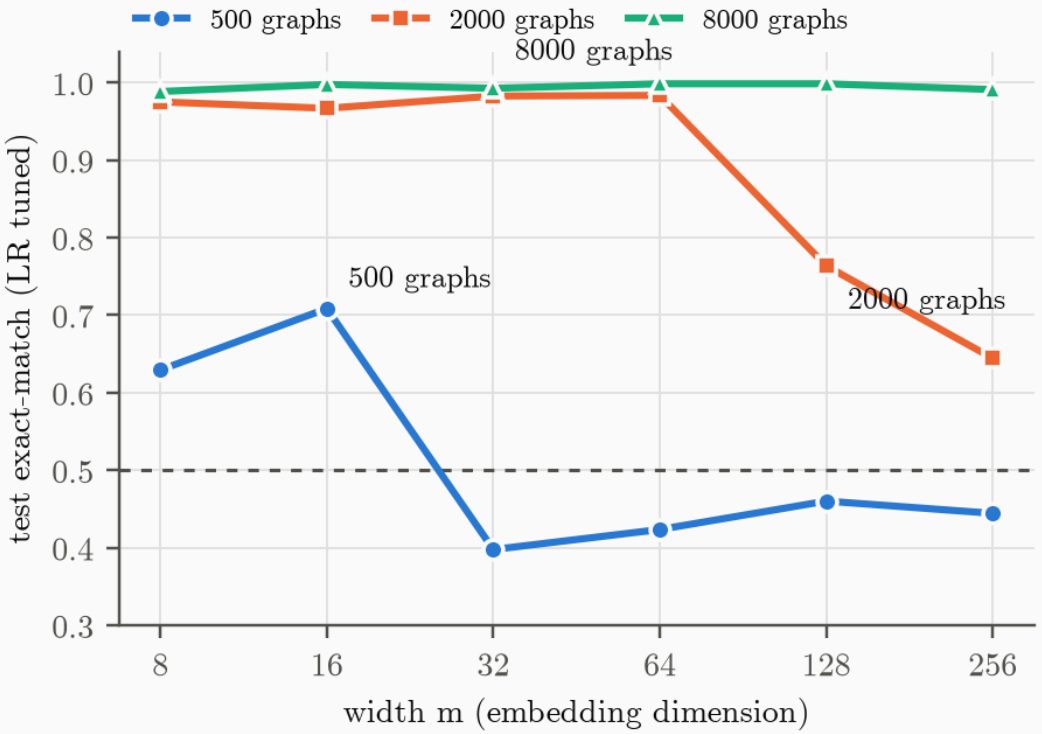
Lesson 1: a fixed learning rate fakes a width effect



- One fixed rate: wide models look up to **0.4 worse**
- Tuned per width: widths 8–64 all ≈ 0.97–0.98
- The best rate **falls as width grows**

$n = 24$, depth 2, 2000 graphs, 3 seeds (Q1/Q1b, 192 runs)

Lesson 2: width has a floor and a data ceiling



- **Floor:** below ≈ 4–8 the model cannot even fit
- **Ceiling:** past it, extra width memorizes
- More data moves the ceiling: at 8000 graphs every $m \geq 16$ reaches ≥ 0.99

same steps for every data size (Q1c, 216 runs)

Lesson 3: the benchmark was leaking

The first dataset could be solved without tracing a single path:

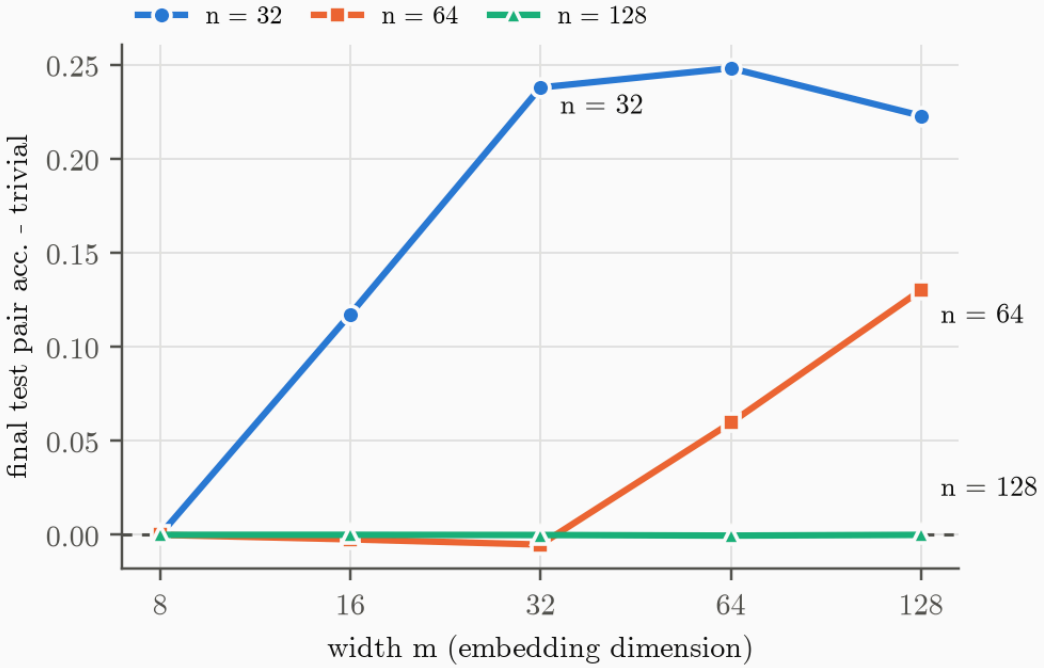
shortcut	**evidence**
node indices reveal the components	blobs always nodes $[0, n_a)$ and $[n_a, n)$ — 200 / 200 graphs
graph statistics reveal the label	degree + short-cycle counts predict it with 0.68–0.90 accuracy
small graphs are local	a 6-hop check is exact on 60–100 % of graphs

Fix — the `swap` generator: the two classes differ only by swapping two far-apart edges (the two-cycles-vs-one "cycle task" of Abbe et al., NeurIPS 2024, plus chords and a dense target); node labels shuffled. Audit: statistics **at chance (0.49–0.53)**, 4-hop checks **score 0**, diameter grows $\approx \log n$.

On the leaky data width 8 sufficed at $n = 24$ — the shortcuts hid the width requirement.

Part 2 — How width must grow

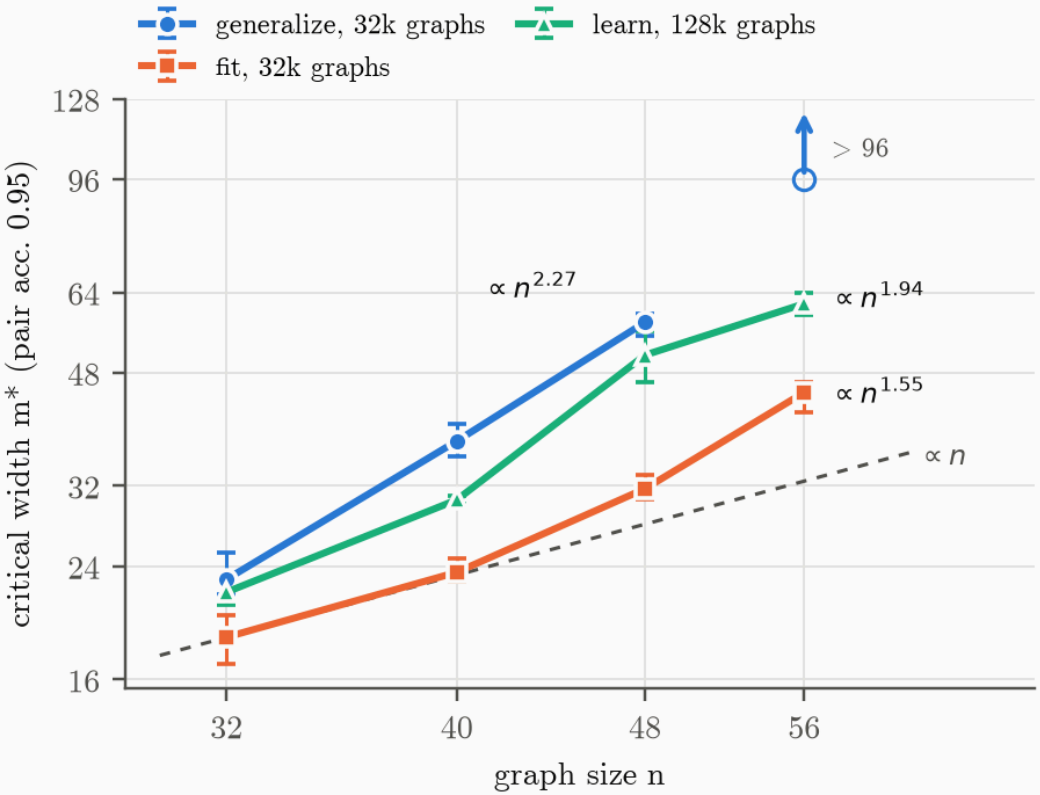
Width decides whether learning starts at all



- $n = 32$: learns from $m \approx 16$
- $n = 64$: only $m \geq 64$ leave the trivial predictor
- $n = 128$: **nothing** up to $m = 128$ learns

depth $\lceil \log_2 n \rceil$, 32 000 graphs, 12 000 steps

Main result: the critical width grows $\approx n^2$



n	m^*	m^*/n
32	21.9	0.68
40	30.2	0.76
48	51.1	1.07
56	57.9	1.03

$m^* \propto n^{1.85}$ (90 % range 1.68–2.03)

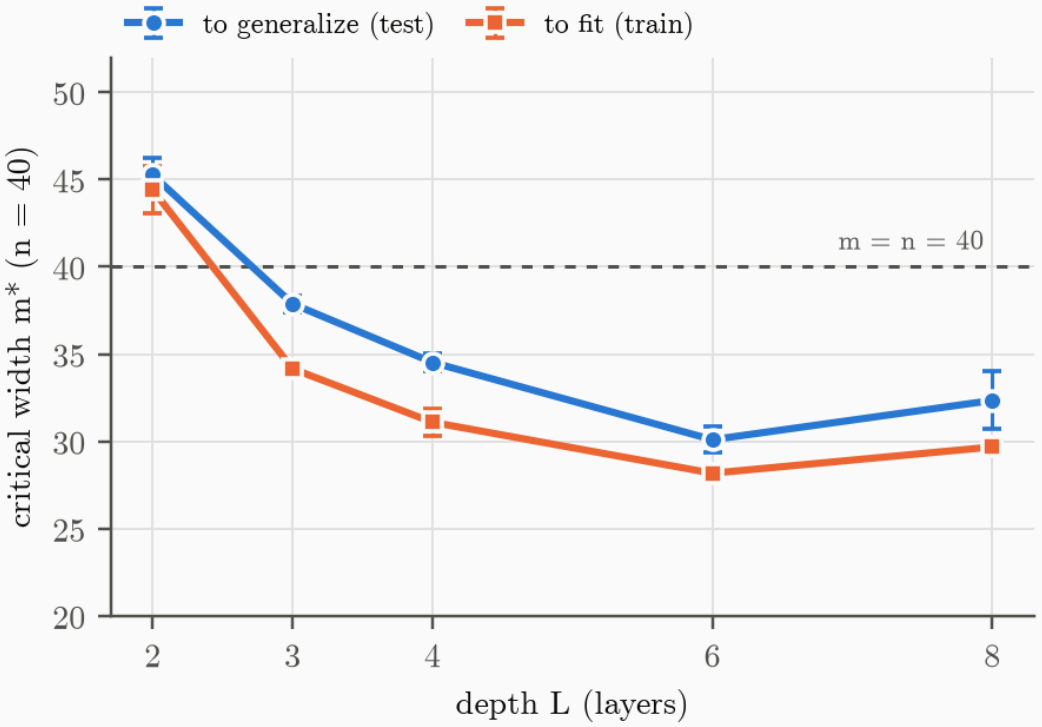
depth 6, 128 000 graphs, 24 000 steps; table: rate tuned at n = 40 / 56 (green line in the chart: one rate, $n^{1.94}$)

The result survives five checks

could it be…	test	outcome
too little data?	32k ⟶ 128k graphs	gap closes, **slope stays ≈ 2**
too little training?	2× the steps	**critical width unchanged**
the learning rate?	second rate, tuned per width	slope 1.94 ⟶ **1.85**
the input encoding?	fixed 96-wide random node IDs	**still ≈ n^2** (2.03)
noise?	2–3 seeds, per-seed crossings	**seeds within ± 3–5**

Over $n = 32$–56, the width a transformer needs to **learn** connectivity grows roughly as n^2 — while theory says this depth needs far less width to **represent** it.

Depth trades for width only mildly



- **2 layers learn** at $m \approx 1.1n$ — on graphs of diameter ≈ 10, so the model does not trace paths hop by hop
- 2 ⟶ 6 layers saves only ≈ ⅓ of the width; 8 = 6
- Matches Yehudai et al. 2025: linear width ⟹ constant depth

$n = 40$, 60 runs

What it means

Contributions so far

1. **A protocol for measuring width.** Learning rate, data and training budget each interact with width enough to fake or hide an effect; we control all three.
2. **An audited benchmark.** Every heuristic we could find sits at or below chance on `swap`; the leaky data had hidden the width requirement.
3. **A measured scaling law.** The critical width to learn connectivity grows $\approx n^{1.9}$ over $n = 32$–56, robust to five confounds.
4. **Representation ≠ learning.** Theory: depth $\log N$ needs width just above $\sqrt{N}$ to represent connectivity. Measured: learning needs $\approx n^2$. Depth helps only mildly.

Limitations

- Graph sizes span less than a factor of two ($n = 32$–56); larger n did not learn at any affordable width
- Two seeds for most scaling points; local slopes vary (≈ 1.2–2.9)
- One task (connectivity) and one architecture family so far
- `swap` diameter grows with $\log n$: path length and node count still rise together
- Compute: Kaggle's 30 GPU-hours per week caps each sweep

Plan for weeks 6–7

item	question	when
Q7c	does the **depth-2** critical width grow linearly or $\approx n^2$?	week 6
Q7b	fix n, vary diameter: is path length or node count driving the growth?	week 6
Q4	Graphormer-style distance bias — on a task it does not give away	weeks 6–7
Q2	task hierarchy: retrieval < connectivity < shortest path	week 7